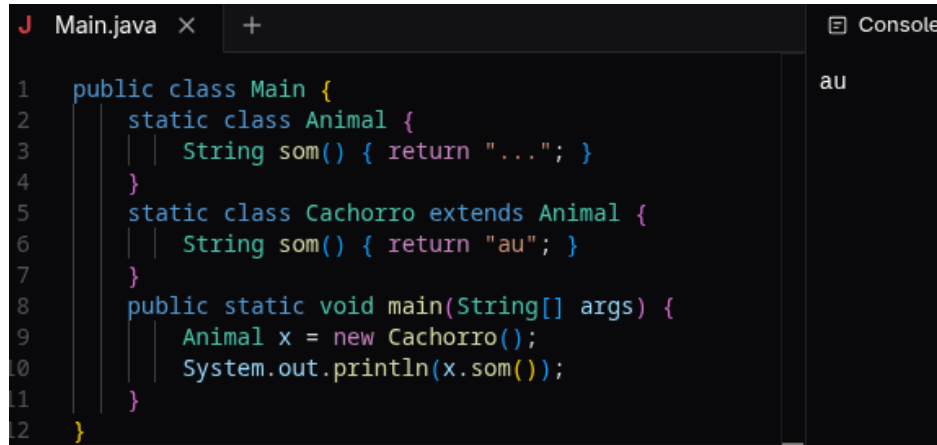


Exercício: qual método é chamado?

Para cada trecho: (a) qual é a saída? (b) a decisão foi tomada na **compilação** ou na **execução**?

1 - Java:

a)



```
1 public class Main {
2     static class Animal {
3         String som() { return "..."; }
4     }
5     static class Cachorro extends Animal {
6         String som() { return "au"; }
7     }
8     public static void main(String[] args) {
9         Animal x = new Cachorro();
10        System.out.println(x.som());
11    }
12 }
```

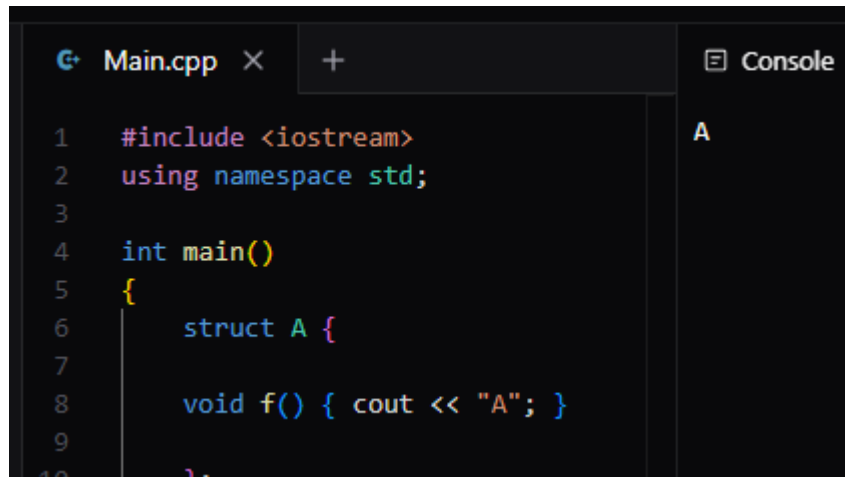
The screenshot shows a Java IDE with a file named 'Main.java'. The code defines a base class 'Animal' with a method 'som()' that returns "...". A subclass 'Cachorro' extends 'Animal' and overrides 'som()' to return "au". The 'main' method creates an instance of 'Cachorro' and calls 'som()'. The 'Console' window on the right shows the output 'au'.

b)

Na execução, pois o método é escolhido pelo tipo do objeto (cachorro).

2 - C++:

a)



```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     struct A {
7
8         void f() { cout << "A"; }
9     }
10 }
```

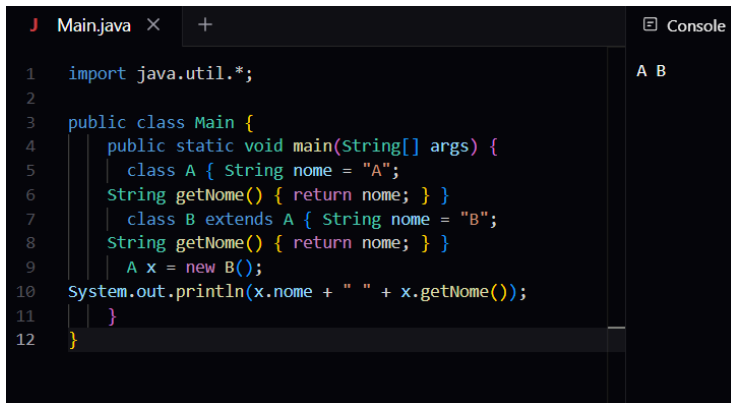
The screenshot shows a C++ IDE with a file named 'Main.cpp'. The code defines a struct 'A' with a non-virtual method 'f()' that prints 'A'. The 'main' function calls 'f()'. The 'Console' window on the right shows the output 'A'.

b)

A decisão foi tomada na compilação, pois a chamada de f() não é virtual, fazendo com que a linguagem use ligação estática

3 - Java:

a)

A screenshot of a Java IDE window titled 'Main.java'. The code defines a class hierarchy: a base class 'A' with a static field 'nome' set to 'A' and a 'getNome()' method; and a subclass 'B' that extends 'A' and overrides 'getNome()' to return 'B'. In the 'main' method, an instance of 'B' is created and its 'getNome()' method is called. The console on the right shows the output 'A B'.

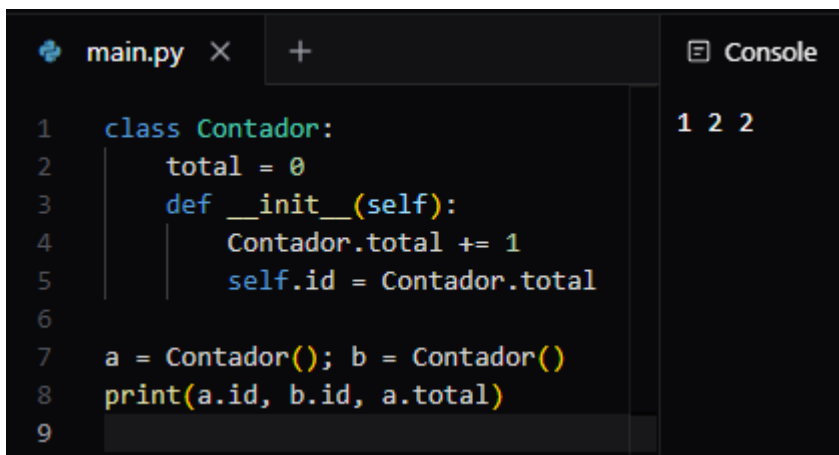
```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         class A { String nome = "A";
6             String getNome() { return nome; } }
7         class B extends A { String nome = "B";
8             String getNome() { return nome; } }
9         A x = new B();
10        System.out.println(x.nome + " " + x.getNome());
11    }
12 }
```

b)

x.nome → resolvido na compilação, pelo tipo declarado (A). Campos não têm polimorfismo, então retorna "A".

x.getNome() → resolvido na execução, pelo tipo real do objeto (B). Métodos sofrem despacho dinâmico, então retorna "B".

4 - Python:

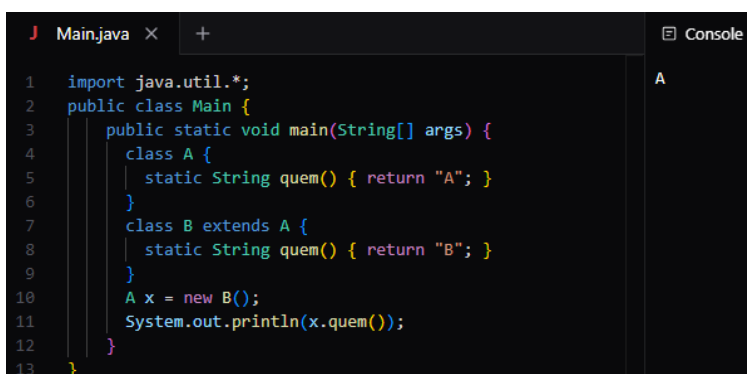
A screenshot of a Python IDE window titled 'main.py'. The code defines a class 'Contador' with a class attribute 'total' set to 0 and an '__init__' method that increments 'total' and sets 'self.id' to the current value of 'total'. Two instances, 'a' and 'b', are created, and their 'id' and 'total' attributes are printed. The console on the right shows the output '1 2 2'.

```
1 class Contador:
2     total = 0
3     def __init__(self):
4         Contador.total += 1
5         self.id = Contador.total
6
7 a = Contador(); b = Contador()
8 print(a.id, b.id, a.total)
9
```

a) Saída: 1 2 2

b) A decisão é feita na Execução do código, ele não consegue pegar as informações antes de executar o código.

5 - Java:

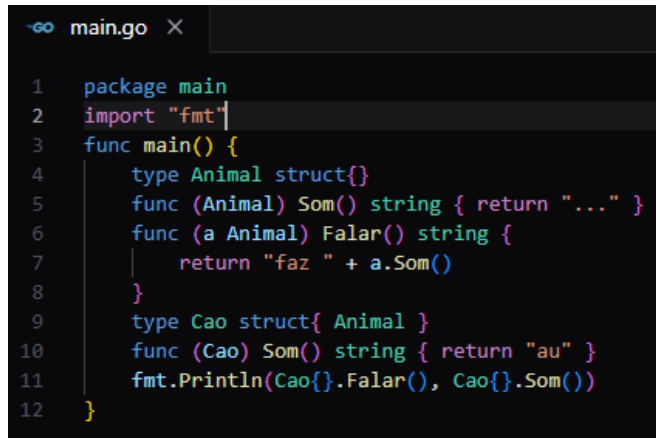
A screenshot of a Java IDE window titled 'Main.java'. The code defines a class hierarchy: a base class 'A' with a static method 'quem()' that returns 'A'; and a subclass 'B' that extends 'A' and overrides 'quem()' to return 'B'. In the 'main' method, an instance of 'B' is created and its 'quem()' method is called. The console on the right shows the output 'A'.

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         class A {
5             static String quem() { return "A"; }
6         }
7         class B extends A {
8             static String quem() { return "B"; }
9         }
10        A x = new B();
11        System.out.println(x.quem());
12    }
13 }
```

a) 'A'

b) A decisão é tomada na compilação

6 - Go:



```
main.go x
1 package main
2 import "fmt"
3 func main() {
4     type Animal struct{}
5     func (Animal) Som() string { return "..."}
6     func (a Animal) Falar() string {
7         return "faz " + a.Som()
8     }
9     type Cao struct{ Animal }
10    func (Cao) Som() string { return "au" }
11    fmt.Println(Cao{}.Falar(), Cao{}.Som())
12 }
```

a) 'faz ... au'

b) A decisão é tomada na compilação